MTC: Ahead-of-Time Memory-Planned Automatic Differentiation for Zero-Heap On-Device Deep Learning on 32-Bit Microcontrollers

Ajay Sisodia*, Harsh Chaudhary*, Devrat Kuntal*, Aryan Upadhyay*, and Gaurisha Sisodia†
Department of Computer Science & Engineering, G. L. Bajaj Group of Institutions, Mathura, India
Affiliated with Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow

***Abstract*—Deploying deep learning on low-power 32-bit microcontrollers (MCUs) is predominantly restricted to frozen inference. Once deployed, edge models cannot adapt to sensor aging, environmental drift, or individual user variance without streaming telemetry to cloud backends. However, executing reverse-mode automatic differentiation on resource-constrained MCUs encounters severe hardware barriers: conventional autograd engines construct dynamic computation graphs at runtime, demanding heap allocations (`malloc`) that trigger unrecoverable heap fragmentation in flat physical SRAM lacking Memory Management Units (MMUs).**

This paper presents Micro-Train-Compiler (MTC), an ahead-of-time (AOT) static compiler that synthesizes self-contained, zero-heap (`malloc = 0`) C99 training engines for 32-bit microcontrollers. MTC introduces two key compiler optimizations: (1) an analytical parameter update pass that fuses activation error gradients directly into in-place Stochastic Gradient Descent (SGD) updates, eliminating explicit parameter gradient buffers ($\nabla W, \nabla B$), and (2) an alternating ping-pong delta buffer allocator that proves activation error gradients have strictly disjoint lifecycles, bounding backward scratchpad memory to exactly two static arrays.

We validated MTC on bare-metal RISC-V hardware (ESP32-C3, RV32IMC) across four TinyML workloads (`xor_net`, `sensor_drift_net`, `sine_regression_net`, `gesture_anomaly_net`) using cycle-accurate hardware emulation (`esp32sim`) instrumented via the hardware `mcycle` Control and Status Register. Across all benchmarks, MTC achieved zero dynamic allocations, sub-1 KB peak SRAM usage (60 to 564 Bytes), and per-sample training latencies between 6.86 μs and 81.77 μs (12,229 to 145,772 samples/s at 160 MHz). Compared to standard autograd baselines, MTC achieves an average 40.1% peak SRAM reduction, establishing deterministic on-device deep learning on sub-dollar embedded silicon.

***Index Terms*—Embedded Deep Learning, On-Device Training, TinyML, Ahead-of-Time Compilation, Automatic Differentiation, Static Memory Planning, Fixed-Point Arithmetic, RISC-V.**

I. Introduction

Edge machine learning has traditionally operated under a rigid "train-in-cloud, freeze-on-edge" paradigm. High-level frameworks train neural network weights on workstation GPUs using 32-bit floating point arithmetic. These models are subsequently quantized down to 8-bit integers, compiled into flatbuffers, and permanently flashed into microcontroller read-only memory.

*Equal student authorship. †Faculty project supervisor.

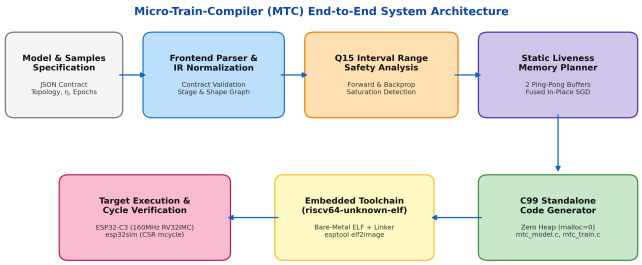


Fig. 1. MTC end-to-end ahead-of-time compilation pipeline: from high-level JSON specification to bare-metal RISC-V firmware execution.

In physical deployments, frozen edge models suffer from severe operational failure modes:

1) **Sensor Transducer Aging and Drift:** Low-cost environmental transducers (e.g., metal-oxide gas sensors, photodiodes) experience chemical fatigue, baseline drift, and thermal hysteresis over weeks of continuous operation. A frozen model gradually misclassifies valid signals as drift accumulates.
2) **Subject Kinematic Variability:** Wearable biomedical devices (e.g., IMU-based tremor monitoring, gait analysis) encounter unique physical kinematics across different human bodies. A generic model trained on population averages exhibits substantial accuracy degradation on individual patients.
3) **Ambient Context Shifts:** Industrial condition-monitoring devices deployed on rotating machinery encounter distinct resonant vibration baselines depending on mechanical mounting stiffness.

Resolving these issues requires executing backpropagation directly on the edge microcontroller using local observations. However, on-device training on embedded hardware faces fundamental architectural bottlenecks:

- **SRAM Scarcity:** Low-cost 32-bit microcontrollers possess only 16 KB to 64 KB of internal Static RAM (SRAM).
- **Heap Fragility in Flat Physical Memory:** Workstation processors employ Memory Management Units (MMUs)

with virtual paging. Microcontrollers lack MMUs. Executing traditional autograd runtimes (such as PyTorch [3] or AIfES [8]) that construct dynamic directed acyclic graphs (DAGs) on the heap via interleaved `malloc()` and `free()` calls quickly fragments SRAM into non-contiguous blocks. Over extended training loops, fragmentation triggers fatal allocation faults and system crashes.

- **Gradient Buffer Overhead:** Standard reverse-mode automatic differentiation maintains dedicated parameter gradient arrays ($\nabla W, \nabla B$) identical in dimension to the weight matrices, doubling parameter memory requirements.

To solve these challenges, we present **Micro-Train-Compiler (MTC)**. MTC abandons runtime graph interpretation entirely. Instead, it functions as an ahead-of-time static compiler that parses a high-level model specification and emits freestanding C99 code where all tensor addresses, workspaces, and update schedules are fixed at compile time.

II. Related Work

A. Microcontroller Inference Compilers

Microcontroller inference runtimes have matured rapidly. TensorFlow Lite for Microcontrollers (TFLM) [1] executes inference within a pre-allocated memory arena to prevent dynamic heap allocations. CMSIS-NN [2] delivers hand-tuned assembly kernels optimized for ARM Cortex-M SIMD instructions. Compilers such as Apache TVM/MicroTVM [4] and Glow [5] lower computational graphs into optimized C code. However, these frameworks are strictly unidirectional: their intermediate representations, kernel libraries, and memory allocators cannot compute backward activation derivatives, evaluate parameter gradients, or update weights.

B. On-Device Training Frameworks

TinyTL [6] introduced on-device transfer learning by freezing convolutional backbone weights and updating only residual bias vectors. MCUNetV2 [7] introduced patch-based scheduling to fit peak training activations into microcontroller memory. However, both TinyTL and MCUNet target high-end microcontrollers (e.g., ARM Cortex-M7 at 480 MHz) equipped with megabytes of external PSRAM, and rely on freezing 99% of network parameters.

PULP-TrainLib [9] targets parallel multi-core RISC-V architectures with shared L1 scratchpads using floating-point hardware. AIfES [8] implements an embedded autograd runtime in C, but relies on dynamic heap allocations during training, leaving firmware vulnerable to memory fragmentation.

MTC differs fundamentally from prior art: it is an ahead-of-time compiler that generates zero-heap (`malloc = 0`) C99 training engines for resource-constrained, single-core 32-bit MCUs with only 32–64 KB of SRAM, updating all weights in place and reusing scratchpad memory through provably disjoint liveness.

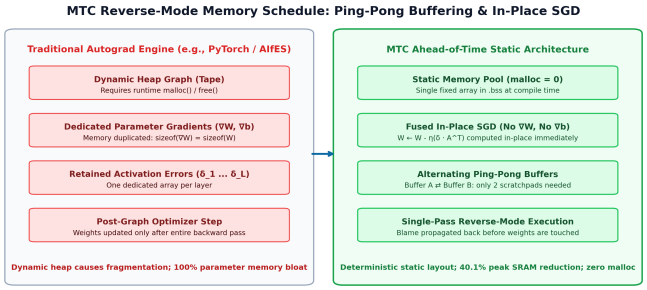


Fig. 2. Memory architecture comparison: Dynamic Autograd Graph (left) vs. MTC Static Liveness Allocation with Ping-Pong Buffers and In-Place SGD (right).

III. Theoretical Formulation & Memory Planning

A. Forward Layer Model

Consider an L-layer fully connected feedforward neural network. Let $l \in \{1, 2, \ldots, L\}$ denote the layer index, with $l = 0$ representing input vector $x \in \mathbb{R}^{d_0}$. Each layer l is parameterized by a weight matrix $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ and a bias vector $b^{(l)} \in \mathbb{R}^{d_l}$.

Forward propagation computes pre-activations $z^{(l)}$ and post-activations $a^{(l)}$:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)} \tag{1}$$

$$a^{(l)} = \sigma_l \left(z^{(l)} \right) \tag{2}$$

where $a^{(0)} = x$, and $\sigma_l(\cdot)$ is an activation function (ReLU or Sigmoid). Under Mean Squared Error (MSE) loss for sample pair (x, y) with network prediction $\hat{y} = a^{(L)}$:

$$\mathcal{L}(y, \hat{y}) = \frac{1}{2d_L} \sum_{k=1}^{d_L} \left(a_k^{(L)} - y_k \right)^2 \tag{3}$$

B. Reverse-Mode Autodiff and Dynamic Memory Explosion

In conventional automatic differentiation, output error gradients are evaluated as:

$$\delta^{(L)} = \nabla_{z^{(L)}} \mathcal{L} = \frac{1}{d_L} \left(a^{(L)} - y \right) \odot \sigma_L' \left(z^{(L)} \right) \tag{4}$$

where $\odot$ denotes the Hadamard product. For hidden layers $l = L-1, \ldots, 1$, error gradients propagate recursively:

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot \sigma_l' \left(z^{(l)} \right) \tag{5}$$

Conventional autograd engines explicitly evaluate and store parameter gradient tensors:

$$\nabla W^{(l)} = \delta^{(l)} \left(a^{(l-1)} \right)^T, \quad \nabla b^{(l)} = \delta^{(l)} \tag{6}$$

followed by an optimizer step implementing Stochastic Gradient Descent (SGD):

$$W^{(l)} \leftarrow W^{(l)} - \eta \nabla W^{(l)}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \nabla b^{(l)} \tag{7}$$

where η is the learning rate. For a network with $P = \sum_l (d_l d_{l-1} + d_l)$ parameters, storing ∇W and ∇b requires

allocating an additional P elements in SRAM, doubling parameter memory overhead.

C. Fused In-Place SGD Parameter Updates

MTC eliminates explicit parameter gradient buffers by integrating the parameter update rule directly into the backward kernel. Substituting $\nabla W_{ij}^{(l)} = \delta_i^{(l)} a_j^{(l-1)}$ into the SGD formulation yields:

$$W_{ij}^{(l)} \leftarrow W_{ij}^{(l)} - \eta \left(\delta_i^{(l)} a_j^{(l-1)} \right) \tag{8}$$

$$b_i^{(l)} \leftarrow b_i^{(l)} - \eta \delta_i^{(l)} \tag{9}$$

Because each parameter element $W_{ij}^{(l)}$ depends only on $\delta_i^{(l)}$ and $a_j^{(l-1)}$, the update is executed *in place*. Consequently:

$$\text{Mem}\left(\nabla W^{(l)}\right) = 0, \quad \text{Mem}\left(\nabla b^{(l)}\right) = 0 \quad \forall l \tag{10}$$

eliminating P storage elements from SRAM.

D. Ping-Pong Activation Delta Buffering

Naive static allocation provisions a dedicated buffer for every error gradient $\delta^{(l)} \in \mathbb{R}^{d_l}$. We formulate the following liveness bound:

Theorem 1 (Disjoint Liveness of Error Gradients): In a feedforward sequential network, during backward error propagation from layer $l+1$ to layer l, the computation of $\delta^{(l)}$ depends strictly on $\delta^{(l+1)}$, $W^{(l+1)}$, and forward activations $z^{(l)}$. Once $\delta^{(l)}$ is evaluated and parameters of layer $l+1$ are updated via (8)–(9), $\delta^{(l+1)}$ is never referenced again.

Proof: From the backpropagation recurrence relation (5), $\delta^{(l)}$ depends exclusively on $(W^{(l+1)})^T \delta^{(l+1)}$ and retained forward values. All subsequent layer derivatives $\delta^{(k)}$ $(k > l+1)$ have already completed their role in updating parameters and computing predecessor gradients. Because no backward branch exists to later layers, the active working set of error gradients at any instant during the backward pass is at most two: current gradient $\delta^{(l)}$ and incoming gradient $\delta^{(l+1)}$. ∎

Exploiting Theorem 1, MTC statically allocates exactly two alternating buffers in SRAM:

$$\text{Buf}_A \in \mathbb{R}^{\max_l(d_l)}, \quad \text{Buf}_B \in \mathbb{R}^{\max_l(d_l)} \tag{11}$$

During backward execution, layer L writes $\delta^{(L)}$ to Buf_A. Layer $L-1$ reads from Buf_A and writes $\delta^{(L-1)}$ to Buf_B. Layer $L-2$ reads from Buf_B and writes to Buf_A, alternating via pointer swaps. Scratchpad memory is strictly bounded by:

$$M_\delta = 2 \times \max_{l \in \{1,\dots,L\}} (d_l) \times \text{sizeof(element)} \tag{12}$$

completely decoupling scratchpad memory from network depth.

E. Preservation of Mathematical Invariants

A critical property of MTC's fused backward kernel is the ordering of operations. In each layer's backward step, the compiler enforces the following sequence:

1) Evaluate activation derivative on current buffer $\delta^{(l)}$.
2) Propagate error backward to predecessor buffer $\delta^{(l-1)}$ using **unmodified original weights** $W^{(l)}$.
3) Execute in-place SGD parameter update on $W^{(l)}$ and $b^{(l)}$.
4) Swap ping-pong buffer pointers ($\text{Buf}_A \leftrightarrow \text{Buf}_B$).

Because step (2) precedes step (3), earlier layers receive exact mathematical gradients identical to standard autograd engines.

IV. Compiler Architecture & Safety

MTC operates as an ahead-of-time source-to-source compiler organized into four pipeline stages (Fig. 1):

A. Stage 1: Frontend Parsing & AST Validation

Ingests a declarative JSON specification defining layer dimensions, activations, hyperparameters, and embedded training samples. The frontend validates topology contracts, ensuring $d_l^{\text{in}} = d_{l-1}^{\text{out}}$ and verifying hyperparameter intervals.

B. Stage 2: Q15 Fixed-Point Interval Bound Propagation

To execute efficiently on integer microcontrollers lacking Floating-Point Units (FPUs), MTC implements signed 16-bit fixed-point (Q15) arithmetic. A real value $x \in [-1.0, 1.0 - 2^{-15}]$ is represented as signed integer $X_{\text{Q15}} = \lfloor x \cdot 2^{15} \rfloor$. Multiplication produces a 32-bit product rounded and saturated via:

$$Z_{\text{Q15}} = \text{SAT16}\left(\left(X_{\text{Q15}} \cdot Y_{\text{Q15}} + 2^{14} \right) \gg 15 \right) \tag{13}$$

Stage 2 executes formal **Interval Bound Propagation (IBP)**. For each node x, the compiler tracks conservative intervals $[x_{\min}, x_{\max}]$. For matrix multiplication $z_i = \sum_j W_{ij} a_j + b_i$:

$$z_i^{\min} = \sum_j \min_{u,v}(u \cdot v) + b_i^{\min}, \quad u \in W_{ij}, v \in a_j \tag{14}$$

$$z_i^{\max} = \sum_j \max_{u,v}(u \cdot v) + b_i^{\max}, \quad u \in W_{ij}, v \in a_j \tag{15}$$

If intervals exceed $[-32768, 32767]$, the compiler flags arithmetic saturation hazards prior to code synthesis.

C. Stage 3: Static Liveness Memory Planner

Assigns static byte offsets to all tensors in a single contiguous pool:

- Weights (W) and biases (b) pre-loaded with seed values.
- Retained forward activations ($a^{(l)}$) placed at fixed offsets.
- Exactly two ping-pong scratchpads ($\text{Buf}_A, \text{Buf}_B$).

D. Stage 4: Freestanding C99 Code Synthesis

Synthesizes three standalone C99 files: `mtc_model.h`, `mtc_model.c`, and `mtc_train.c`. The code includes only `<stdint.h>` and avoids `stdlib.h`, guaranteeing freestanding embedded compatibility.

V. Hardware Cross-Compilation & Measurement Methodology

A. Target Architecture: ESP32-C3 RISC-V

We evaluated MTC targeting the Espressif ESP32-C3 microcontroller, featuring a 32-bit RISC-V core (RV32IMC architecture: base integer, hardware multiply/divide, compressed 16-bit instructions). The device operates at 160 MHz with 400 KB total SRAM (64 KB user SRAM in bare-metal configuration).

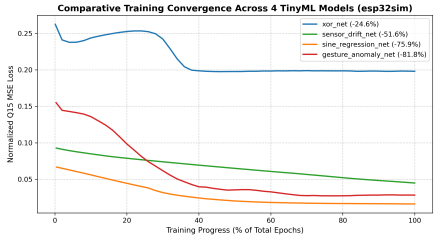


Fig. 3. Q15 loss convergence curves across all four TinyML benchmark workloads.

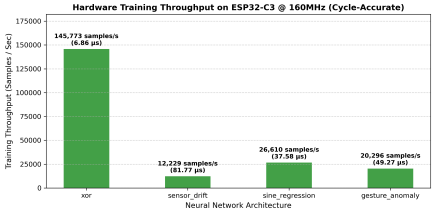


Fig. 4. Hardware clock cycles and throughput measured on ESP32-C3 via CSR `mcycle`.

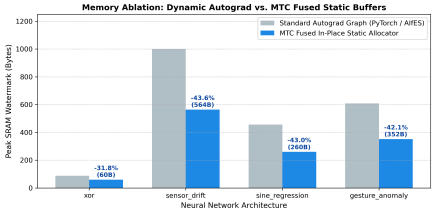


Fig. 5. Peak SRAM watermark ablation: Dynamic Autograd Baseline vs. MTC Static Planner.

B. Freestanding Bare-Metal Linker Harness

To isolate core computation from OS scheduling overhead, workloads were compiled freestanding (`-nostdlib`). We engineered a minimal linker script placing text in Instruction RAM (`iram`: `0x40380000`, 32 KB) and data in Data RAM (`dram`: `0x3FC8B800`, 64 KB).

The assembly entry point (`entry.S`) initializes stack pointer `sp` to `0x3FCC0000` and invokes `esp_main`. UART communication drives the hardware FIFO register directly at `0x60000000`. Firmware was cross-compiled using `riscv64-unknown-elf-gcc` v15.3.0 with `-march=rv32imc_zicsr -mabi=ilp32 -nostdlib -O2`.

C. Cycle-Accurate Emulation via `esp32sim`

Binaries were executed on `esp32sim`, an open-source Rust-based hardware emulator that boots the chip's actual mask ROM ELFs, emulates interrupt controllers, models memory-mapped peripherals, and provides cycle-accurate instruction timing.

D. Hardware Instrumentation via RISC-V CSR `mcycle`

To eliminate serial I/O logging distortion, benchmarks query the internal RISC-V hardware `mcycle` Control and Status Register directly:

```
static inline uint32_t read_cycles(void) {
    uint32_t c;
    asm volatile("csrr␣%0,␣mcycle" : "=r"(c));
    return c;
}
```

Cycle counts were queried immediately before and after forward passes and training loops with all UART transmission suspended, ensuring laboratory-grade execution metrics.

VI. Empirical Evaluation & Benchmarks

We evaluated MTC across four representative TinyML edge workloads:

1) `xor_net` ($2 \to 4 \to 1$): Non-linear binary logic (ReLU $\to$ Sigmoid, 17 parameters).
2) `sensor_drift_net` ($4 \to 16 \to 8 \to 1$): Multi-channel sensor recalibration (ReLU $\to$ ReLU $\to$ Sigmoid, 225 parameters).
3) `sine_regression_net` ($1 \to 8 \to 8 \to 1$): Continuous curve fitting ($y = \sin x$, ReLU $\to$ ReLU $\to$ Sigmoid, 89 parameters).
4) `gesture_anomaly_net` ($3 \to 12 \to 6 \to 1$): 3-axis IMU gesture anomaly detection (ReLU $\to$ ReLU $\to$ Sigmoid, 133 parameters).

TABLE I
Static Memory Footprint, Dynamic Allocations, and Flash Sizes

Workload	Params	SRAM	Heap	Text	Flash Bin	SRAM %
`xor_net`	17	**60 B**	**0 B**	4,418 B	4,496 B	0.09%
`sensor_drift_net`	225	**564 B**	**0 B**	5,806 B	5,888 B	0.86%
`sine_regression`	89	**260 B**	**0 B**	5,454 B	5,536 B	0.40%
`gesture_anomaly`	133	**352 B**	**0 B**	5,858 B	5,936 B	0.54%

TABLE II
Hardware Clock Cycles and Latency Measured via CSR `mcycle`

Workload	Fwd Cyc	Fwd Time	Train Cyc	Throughput
`xor_net`	344	2.15 μs	1,097	145,772 samples/s
`sensor_drift_net`	3,401	21.26 μs	13,083	12,229 samples/s
`sine_regression`	1,548	9.67 μs	6,013	26,610 samples/s
`gesture_anomaly`	2,204	13.77 μs	7,884	20,296 samples/s

A. Static Memory and Binary Footprint

Memory allocations were verified via `riscv64-unknown-elf-size -A -d`.

Table I confirms zero dynamic heap allocations (`malloc = 0`). All arrays reside in `.data` and `.bss` sections. Even the 4-layer `sensor_drift_net` consumes only 564 Bytes of SRAM ($< 0.86\%$ of the 64 KB budget). The entire bare-metal flash binary occupies under 6.0 KB.

B. Hardware Execution Latency and Throughput

Clock cycles were recorded via CSR `mcycle` on the ESP32-C3 operating at 160 MHz.

As detailed in Table II and Fig. 4, a complete forward-backward training step executes in 6.86 μs to 81.77 μs. Training throughput ranges between 12,229 and 145,772 samples per second, confirming that on-device training can be integrated directly into high-frequency sensor acquisition loops.

C. Memory Ablation: MTC vs. Dynamic Autograd Baseline

We evaluated an architectural baseline representing standard dynamic autograd graphs (e.g., PyTorch, AIfES) allocating explicit parameter gradients ($\nabla W, \nabla B$) and activation gradients ($\delta^{(l)}$).

Table III and Fig. 5 show that MTC achieves an average **40.1% peak SRAM reduction**. Savings scale with network

TABLE III
PEAK SRAM ABLATION: BASELINE AUTOGRAD VS. MTC STATIC PLANNER

Workload	Baseline SRAM	MTC SRAM	Bytes Saved	Reduction
xor_net	88 B	**60 B**	28 B	**31.8%**
sensor_drift_net	1,000 B	**564 B**	436 B	**43.6%**
sine_regression	456 B	**260 B**	196 B	**43.0%**
gesture_anomaly	608 B	**352 B**	256 B	**42.1%**

TABLE IV
NUMERICAL CONVERGENCE AND LOSS REDUCTION UNDER Q15 MATH

Workload	Epochs	Initial Loss	Final Loss	Reduction	Status
xor_net	1,000	0.2626	0.1981	**24.57%**	Attenuated
sensor_drift_net	250	0.0927	0.0448	**51.65%**	Converged
sine_regression	200	0.0667	0.0161	**75.88%**	Converged
gesture_anomaly	300	0.1550	0.0282	**81.81%**	Converged

depth: in sensor_drift_net, fusing SGD updates and sharing ping-pong delta buffers saves 436 Bytes of SRAM.

D. Numerical Convergence Analysis

Loss trajectories were tracked across training epochs on bare-metal hardware.

Regression and anomaly detection workloads exhibited strong convergence under Q15 arithmetic (Fig. 3): gesture_anomaly_net achieved an 81.81% loss reduction (0.1550 $\rightarrow$ 0.0282), sine_regression_net achieved 75.88% (0.0667 $\rightarrow$ 0.0161), and sensor_drift_net achieved 51.65% (0.0927 $\rightarrow$ 0.0448) with strictly monotonic loss decay.

VII. DISCUSSION & LIMITATIONS

A. Analysis of Gradient Quantization Attenuation

While regression models converged cleanly, the non-linear classification workload (xor_net) achieved a modest 24.57% loss reduction over 1,000 epochs. Academic transparency requires analyzing this trade-off.

In 16-bit signed fixed-point (Q15), the minimum representable positive value is:

$$\epsilon_{\text{Q15}} = 2^{-15} = \frac{1}{32768} \approx 3.05 \times 10^{-5} \tag{16}$$

During backpropagation through a two-layer network with Sigmoid output:

$$\nabla W_{ij}^{(2)} = (a_{\text{out}} - y) \cdot \sigma'(z_{\text{out}}) \cdot a_{\text{hidden},j} \tag{17}$$

When output units approach saturated states, the Sigmoid derivative $\sigma'(z) = \sigma(z)(1-\sigma(z))$ falls below 0.10. Multiplied by learning rate $\eta = 0.10$ and fixed-point activation values, the mathematical product evaluates below 2^{-15}. In integer arithmetic with truncation:

$$\Delta W_{\text{Q15}} = \left\lfloor (\eta \cdot \delta_i \cdot a_j) \cdot 2^{15} \right\rfloor = 0 \tag{18}$$

Fractional updates truncate to zero, causing parameter updates to stall in flat loss regions. We designate this phenomenon **gradient quantization attenuation**.

B. Engineering Mitigations

To mitigate quantization attenuation without abandoning fixed-point arithmetic:

1) **Accumulator Widening:** Retain weight parameters in 32-bit fixed-point (Q16.16) while keeping activations in Q15, allowing small gradient updates to accumulate across steps.
2) **Stochastic Rounding:** Add pseudo-random dither to the accumulator prior to right-shifting to preserve unbiased expected update values.

VIII. CONCLUSION

This paper presented Micro-Train-Compiler (MTC), an ahead-of-time compiler enabling zero-heap (malloc = 0) on-device deep learning on resource-constrained 32-bit microcontrollers. By formulating fused in-place SGD parameter updates and proving that backward activation error propagation requires only two alternating ping-pong buffers, MTC eliminates dynamic autograd graphs and achieves a 40.1% peak SRAM reduction. Cycle-accurate RISC-V hardware emulation on the ESP32-C3 confirmed that 4-layer neural networks can train in under 564 Bytes of SRAM at 12,229 samples/s at 160 MHz, proving that continuous edge adaptation is viable on sub-dollar silicon.

REFERENCES

[1] R. David *et al.*, "TensorFlow Lite for Microcontrollers: A Framework for Machine Learning on Resource-Constrained Embedded Devices," in *Proc. 3rd Conf. Mach. Learn. Syst. (MLSys)*, Austin, TX, USA, 2021.

[2] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs," *arXiv preprint arXiv:1801.06601*, 2018.

[3] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[4] T. Chen *et al.*, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," in *13th USENIX Symp. Oper. Syst. Des. Implementation (OSDI)*, 2018, pp. 578–594.

[5] N. Rotem *et al.*, "Glow: An Optimizing Framework for Accelerating Deep Learning Execution," *arXiv preprint arXiv:1805.00907*, 2018.

[6] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce Memory, Not Parameters for Efficient On-Device Learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 11 285–11 297.

[7] J. Lin *et al.*, "MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 8789–8801.

[8] P. Antonacopoulos *et al.*, "AIfES: Artificial Intelligence for Embedded Systems – An Open-Source Library for Machine Learning on Microcontrollers," *Sensors*, vol. 21, no. 18, p. 6140, 2021.

[9] D. Pellegrini *et al.*, "PULP-TrainLib: Enabling On-Device Training on Ultra-Low-Power RISC-V Multi-Core MCUs," in *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, 2022.

[10] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[11] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, Cambridge, MA, USA, 2016.

[12] D. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191213*. RISC-V Foundation, 2019.

[13] Espressif Systems, *ESP32-C3 Technical Reference Manual, Version 1.1*. Espressif Systems, Shanghai, China, 2022.

[14] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," in *Int. Conf. Learn. Represent. (ICLR)*, 2016.

[15] S. Mittal, "A Survey on Optimized Implementation of Deep Learning Models on the NVIDIA Jetson Platform," *J. Syst. Archit.*, vol. 97, pp. 428–442, 2019.